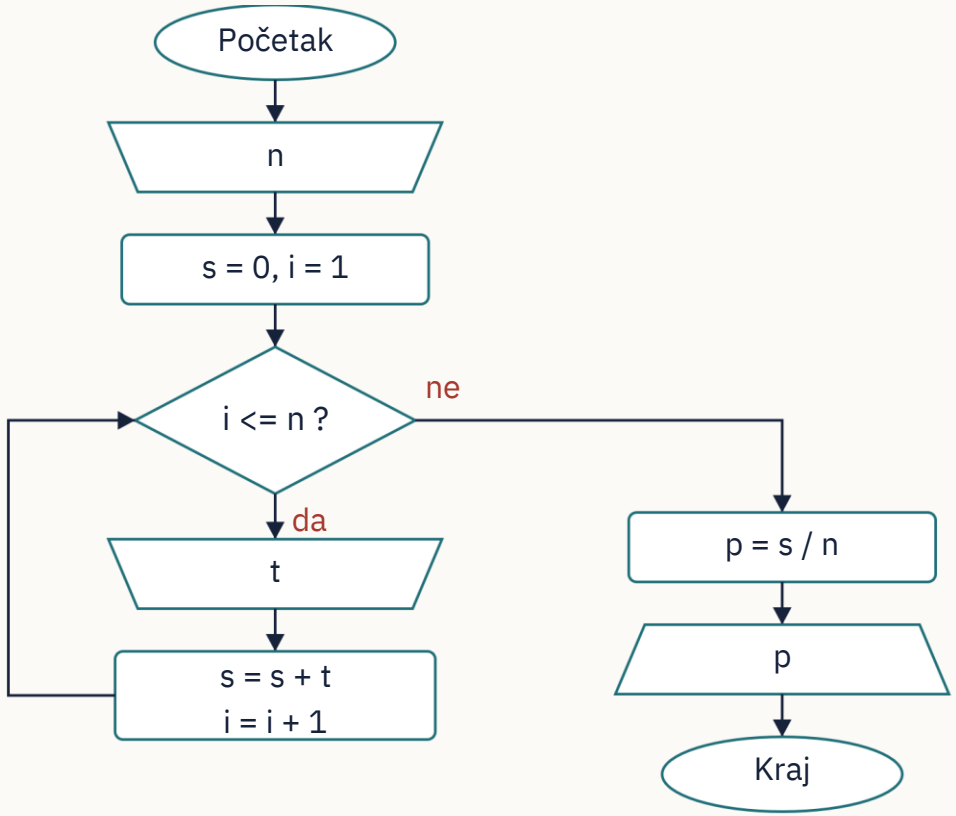
PROGRAMIRANJE · MEHATRONIKA, I GODINA, I SEMESTAR

While petlja i ponavljanje

program koji ponavlja (pravi petlju)

Predavanje 3

Šta nam je novo na ovom dijagramu



Postavka: Uređaj očitava n mjerenja temperature i računa njihovu srednju vrijednost.

Pisaćemo program za ovaj dijagram

Strelica nazad

U dijagramu se tok **vraća** na romb. Treba nam naredba koja **ponavlja** dio programa: to je **while**.

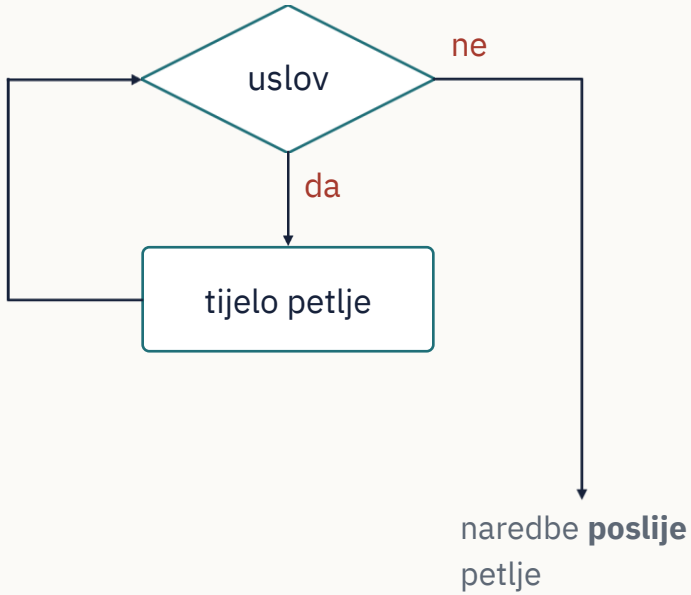
Brojač

s je zbir očitanih vrijednosti, ***i*** je brojač koji pamti koliko ih je do sada očitano, tj. ***i*** broji prolaze ili iteracije petlje. I ***s*** i ***i*** dobijaju početnu vrijednost prije ulaska u petlju.

Uslov izlaska

Petlja mora jednom stati. Uslov **i <= n** postaje netačan tačno onda kada su sva mjerenja učitana.

Naredba while ili while petlja




```
while (uslov)
{
   naredba;
   naredba;
}
naredbe poslije petlje
```

tijelo petlje

Prvo definišemo uslov

While petlja provjerava uslov **prije** svakog prolaza. Ako je netačan već na početku, tijelo se ne izvrši nijednom.

Poslije petlje

Kada uslov postane netačan, program nastavlja prvom naredbom **iza** zatvorene zagrade.

U jeziku C petlju prave tri naredbe: **while**, **do-while** i **for**. Danas radimo **while**.

Pratite petlju prolaz po prolaz

1. popunjavamo tabelu na papiru, bez računara
2. pišemo program

```
int p = 2;

while (p <= 1000)
   p = 2 * p;

printf("%d\n", p);
```

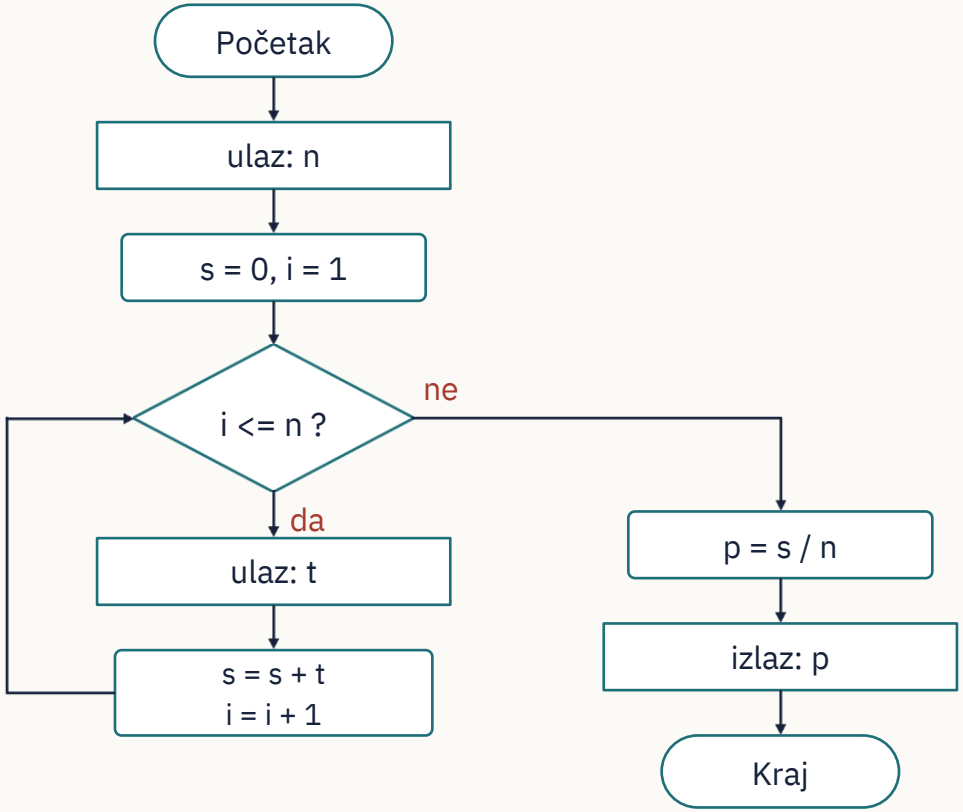
Prolaz	p prije uslova	p <= 1000	p poslije prolaza
1	2	1	4
2	4	1	8
3			
4			
...			
kraj		0	—

Kada tijelo petlje čini samo **jedna** naredba, vitičaste zagrade nisu neophodne (ali nije greška ako ih dodamo)

Prije nego što pokrenete

Koliko puta se izvrši naredba **p = 2 * p**? Šta ispisuje **printf**?

Dijagram 3 postaje program



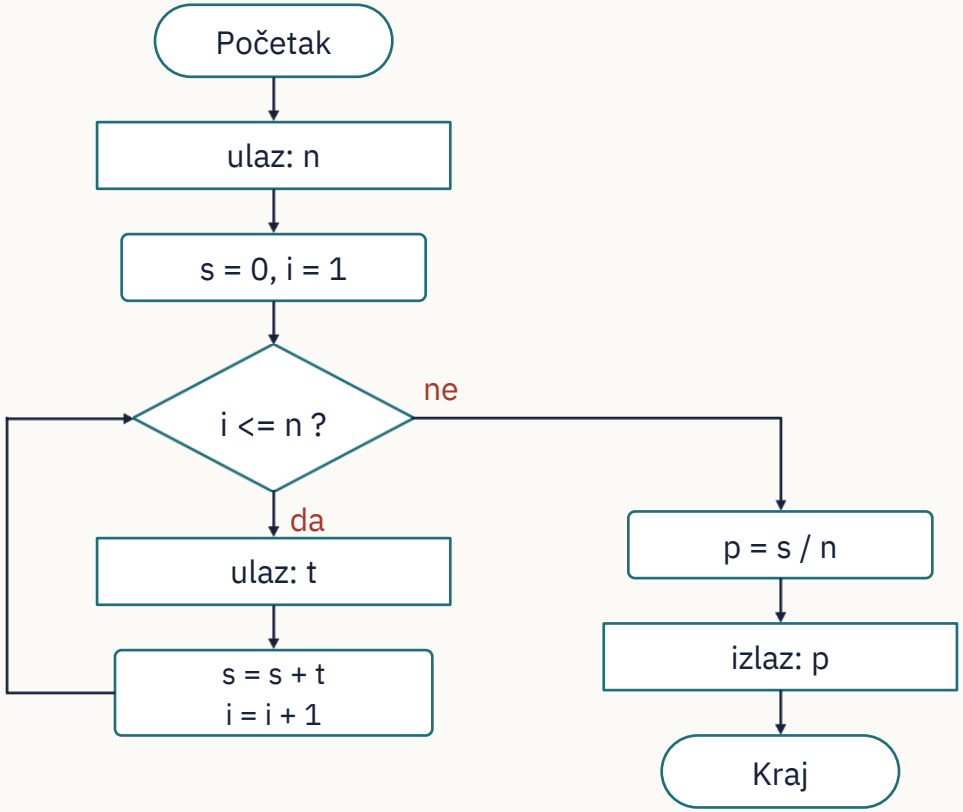

```
int n, i;
float t, s, p;

printf(„Unesite broj mjerenja: ");
scanf("%d", &n);
s = ___;
i = ___;
while (_____________) {
   printf(„Unesite mjerenje broj %d: ", i);
   scanf("%f", &t);
   s = _____________;
   i = _____________;
}
p = _____________;
printf("Srednja vrijednost: %.1f\n", p);
```

Provjera

Za n = 3 i mjerenja 20, 22 i 24 program mora ispisati **22.0**.

Dijagram 3 postaje program rješenje




```
int n, i;
float t, s, p;

printf(„Unesite broj mjerenja: ");
scanf("%d", &n);
s = 0;
i = 1;
while (i <= n) {
   printf(„Unesite mjerenje broj %d: ", i);
   scanf("%f", &t);
   s = s + t;
   i = i + 1;
}
p = s / n;
printf("Srednja vrijednost: %.1f\n", p);
```

Pitanje

Šta bi program ispisao da je **s** deklarisano kao **int**, a mjerenja su 20.5, 21 i 22? Testirajte

Beskonačna petlja - petlja koja se ne zaustavlja

pokrenite u Code::Blocks-u

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
}
```

Prevodilac neće prijaviti grešku

Program je ispravno napisan u jeziku C, ali algoritam nije dobro napisan. Ovakvu grešku otkrićemo tek kada pokrenemo program.

Kako da prekinemo beskonačnu petlju

Tasteri **Ctrl+C** u konzoli ili zatvaranje prozora konzole.

Vaš zadatak

Dodajte jedan red tako da program ispiše brojeve od 1 do 5 i stane. Gdje tačno ide taj red? Šta se ispiše ako ga stavite iznad **printf**?

Kraći zapis dodjele

x prije	Kraći zapis	Duži zapis	x poslije
10	x += 3	x = x + 3	13
10	x -= 4	x = x - 4	6
10	x *= 2	x = x * 2	20
10	x /= 4	x = x / 4	2
10	x %= 3	x = x % 3	1
10	x++	x = x + 1	11
10	x--	x = x - 1	9

Prepišite petlju iz programa za srednju vrijednost koristeći operator **+=** i operator **++**. Rezultat mora ostati isti.

Pazite na redoslijed znakova

x =+ 3 se prevodi bez greške, ali ne znači uvećanje za 3 već znači **x = +3, tj.** x postaje 3.

Ništa novo ne računa, samo se kraće piše. Program radi isto i sa kraćim i sa dužim zapisom.

Pogoditi ispis

zapišite četiri broja prije pokretanja

```
int c = 5;
printf("%d\n", c++);
printf("%d\n", c);

c = 5;
printf("%d\n", ++c);
printf("%d\n", c);
```

ispis 1	ispis 2	ispis 3	ispis 4
?	?	?	?

Pogoditi ispis

```c
int c = 5;
printf("%d\n", c++);
printf("%d\n", c);

c = 5;
printf("%d\n", ++c);
printf("%d\n", c);
```

ispis 1	ispis 2	ispis 3	ispis 4
5	6	6	6

Poslije pokretanja

Oba puta **c** postaje 6. Zašto se onda prvi i treći ispis razlikuju?

++i je prefiksni a i++ je postfiksni operator inkrementiranja

U petlji pišemo **i++;** kao **zasebnu naredbu**. Tada je svejedno da li je **++** ispred ili iza **i**. Primjer:

```c
i = 1;
while (i <= n) {
   printf(„Unesite mjerenje broj %d: ", i);
   scanf("%f", &t);
   s = s + t;
   i++;
}
```

Kada broj mjerenja nije poznat

sentinel vrijednost

Novi zadatak

Tehničar unosi mjerenja jedno po jedno i **ne zna unaprijed** koliko ih ima. Dakle, broja **n** nema u ovom primjeru.

Sentinel

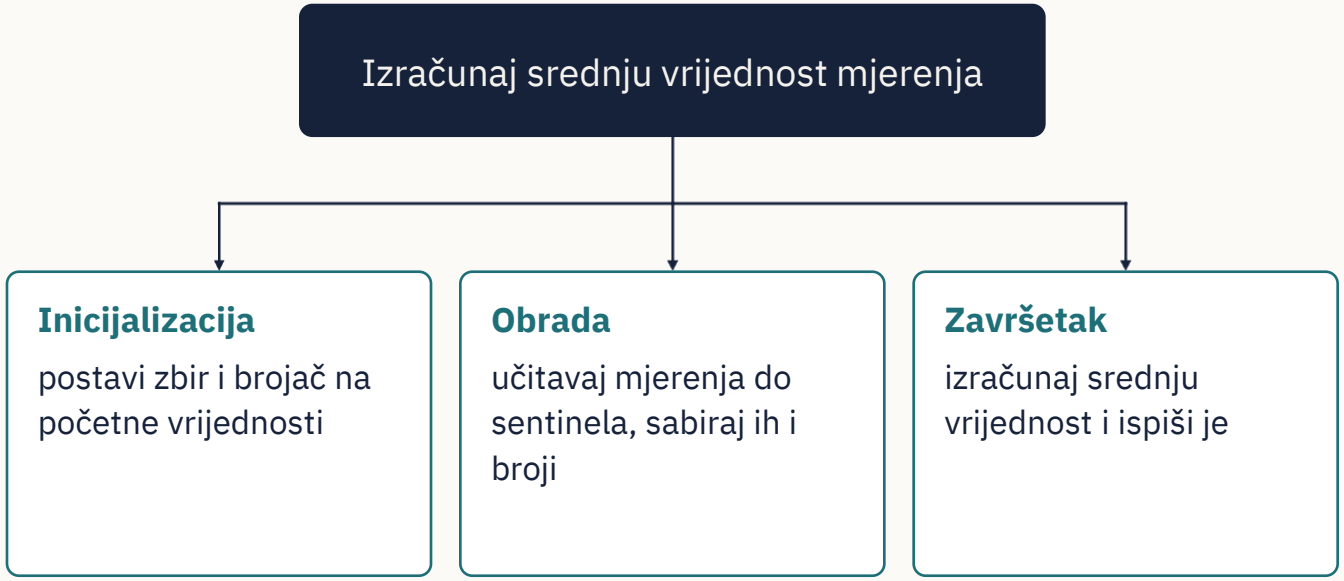
Dogovorena vrijednost koja ne može biti stvarni podatak. Kada je program pročita, unos je završen.

Pitanje

Može li **-1** biti izmjerena vrijednost za temperaturu? Može. Zato za sentinel predložite vrijednost koja sigurno ne može biti izmjerena. Npr. -999

Sentinel se ne sabira i ne broji. On samo zaustavlja petlju.

Razlaganje problema odozgo nadolje



Pseudokod

Algoritam zapisan rečenicama ljudskog jezika, sa uvlačenjem kao u kodu. Ne prevodi se i služi da se problem riješi prije kucanja.

Vaš zadatak

Razložite svaku fazu na korake u pseudokodu. Kraj unosa označava **-999**.

Skoro svaki program ima ove tri faze. Krupan zadatak dijelimo dok svaki korak ne postane jedan red koda.

Od pseudokoda do C koda

kraj unosa je -999

Pseudokod

```
zbir = 0, brojač = 0
učitaj mjerenje
dok mjerenje nije -999
    dodaj mjerenje zbiru
    povećaj brojač za 1
    učitaj mjerenje
ako brojač nije 0
    ispiši zbir / brojač
inače
    ispiši „niste unijeli nijedno mjerenje"
```

```c
float t, s = 0;
int i = 0;

printf("Mjerenje (-999 za kraj): ");
scanf("%f", &t);
while (t != -999) {
    s += t;
    i++;
    printf("Mjerenje (-999 za kraj): ");
    scanf("%f", &t);
}
if (i != 0)
    printf("Srednja vrijednost: %.1f\n", s / i);
else
    printf(" niste unijeli nijedno mjerenje \n");
```

Mjerenje se učitava na dva mjesta

Prvo **prije petlje**, da bi uslov imao šta da provjeri. Svako sljedeće mjerenje se učitava **na kraju tijela petlje**, za naredni prolaz petlje (narednu iteraciju).

Više od dvije grane: else if

```
int b; // bodovi
char o; // ocjena

printf("Bodovi: ");
scanf("%d", &b);
if (b >= 90)
   o = 'A';
else if (b >= 80)
   o = 'B';
else if (_____)
   o = ___;
...
else
   o = 'F';
printf("Ocjena: %c\n", o);
```

A ≥ 90 | B ≥ 80 | C ≥ 70 | D ≥ 60 | E ≥ 50 | F < 50

Uslovi se provjeravaju redom

Prvi tačan uslov izvršava svoju granu, a ostali se preskaču. Zato za ocjenu B nije potrebno pisati **b >= 80 && b < 90**

else if nije nova naredba

To je **if** u grani **else**, zapisan u istom redu (razlog je čitljivost koda, da kod ne bi išao udesno previše). Prevodilac ne gleda uvlačenje; uvlačimo zbog čovjeka koji čita kod.

Probajte

Premjestite granu za E (**b >= 50**) na prvo mjesto. Koju ocjenu sada dobija student sa 95 bodova? Redoslijed uslova je **dio algoritma.**

Odluka unutar petlje: kontrola kvaliteta

Postavka

Na traci prolazi serija od 10 komada. Za svaki komad operater unosi **1** ako je ispravan, a **2** ako je škart.

Program na kraju ispisuje broj ispravnih komada, broj škart komada i procenat ispravnih. Ako je ispravnih više od 8, ispisuje **Serija prihvaćena**.

Pitanje 1

Koliko brojača treba programu i šta broji svaki od njih?

Pitanje 2

Koja odluka ide unutar petlje, a koja poslije nje?

Pitanje 3

Kako dobiti procenat kada su oba broja cijela?

Prvo pravimo tri faze u pseudokodu, pa tek onda kod.

Kontrola kvaliteta u kodu

rješenje

```
int r, isp = 0, skart = 0, k = 1;

while (k <= 10) {
    printf("Komad %d (1 ispravan, 2 skart): ", k);
    scanf("%d", &r);
    if (r == 1)
        isp++;
    else
        skart++;
    k++;
}
printf("Ispravnih: %d, skart: %d\n", isp, skart);
printf("Procenat: %.1f\n", (float) isp / 10 * 100);
if (isp > 8)
    printf("Serija prihvacena\n");
```

Odluka u petlji (if-else komanda)

Za svaki komad bira koji brojač raste. Izvršava se **deset puta**.

Odluka poslije petlje (if komanda)

Ocjenjuje cijelu seriju. Izvršava se **jednom**.

Konverzija tipa

(float) isp pretvara cio broj u realan samo za ovaj račun. Bez toga **9 / 10** daje 0, pa je i procenat 0.

Tri greške u petlji koje prevodilac ne prijavljuje

Tačka-zarez iza uslova proizvodi beskonačnu petlju

```
while (i <= n);
{
    s += t;
    i++;
}
```

Petlja nema tijelo, a **i** se u njoj ne mijenja. Program stoji i ništa ne ispisuje.

Tijelo bez zagrada

```
while (i <= n)
    printf("%d\n", i);
    i++;
```

Petlji pripada samo prvi red. Red **i++** se nikad ne izvrši, iako je uvučen.

Zbir bez početne vrijednosti

```
float s;
...
s += t;
```

s počinje od vrijednosti koja je slučajno bila u memoriji. Rezultat je besmislen.

Kako provjeriti petlju

Pokrenite program za n = 0, n = 1 i n = 2. Greške u petlji najčešće se vide baš na tim vrijednostima.

Zadaci

Zadatak 1

Učitati **n** i ispisati brojeve od 1 do **n**.

Zadatak 2

Učitavati temperature do unosa -999. Ispisati ALARM kada tri uzastopna mjerenja pređu 80 stepeni.

Zadatak 3

Učitavati cijele brojeve do unosa 0 i ispisati najveći od njih.

Zadatak 4

Učitati **n** i ispisati zbir parnih brojeva od 1 do **n**. Koristiti **%** i **+=**.